D2 - Testing and 2nd release of the integrated scheduler

Project name
ANDREAS

Agreement Number

Duration
M6 - M8

Deliverable reference
D2

Date of delivery
M8

Responsible partner
7bulls.com

Editor and authors
Paweł Skrzypek (editor), Katarzyna Materka, Federica Filippini, Fabrizio Magugliani, Maciej Riedl

Reviewers
Danilo Ardagna

Distribution
Confidential

Executive Summary

This deliverable describes the second release of the GPU scheduler developed within the ANDREAS project and includes a high level description of the ANDREAS solution architecture, the details of the resource management scheduling algorithm implemented and its preliminary performance evaluation. Also the results of some preliminary tests are provided.

This deliverable is the results of the work achieved within task *T2 - EUBra-BigSEA scheduler integration with the SLURM queue* during the first eight months of the project.

Preliminary results have shown how the scheduling policies are effective providing a cost reduction in different environments (on-premises and cloud clusters) .

D2 - Testing and 2nd release of the integrated scheduler **1**
Executive Summary 1
1. Introduction **3**
2. Architecture changes in 2nd release **3**
3. Integration with SLURM **6**
4. ARMIDA software updates **6**
5. Machine learning jobs docker image with GPU support **7**
6. Jobs optimizer changes in 2nd release **7**
6.1 New proxy function 7
6.2 Path Relinking algorithm 8
6.2.1 Optimizer preliminary performance evaluation 11
7. Testing results **14**
8. Conclusion **16**
9. References **16**

1. Introduction

This deliverable describes the second release of the GPU scheduler developed within the ANDREAS project and includes changes introduced in the second release compared to the first release (described in deliverable D1). This deliverable is the results of the work achieved within task *T2 - EUBra-BigSEA scheduler integration with the SLURM queue* during the first eight months of the project.

This document is organized as follows. The next section introduces the changes in high level architecture of ANDREAS scheduler. Section 3 describes the integration with SLURM framework on the experimental ARMIDA cluster available at E4 company, while Section 4 provides further details related to software updates on the ARMIDA cluster. Section 5 contains a description of the base docker image for machine learning jobs with Tensorflow for ARM architecture used in the ANDREAS project. Section 6 provides a description of the changes in algorithms used by the Jobs Optimizer component which implements the intelligence to optimize the training job's placement while the testing process we envision and the evaluation is reported in Section 7. Conclusions, ongoing and future works are finally drawn in Section 8.

2. Architecture changes in 2nd release

This section presents the high-level architecture of the solution created within the ANDREAS project, along with the changes compared to preliminary architecture described in deliverable D1 "D1 - 1st release of the integrated scheduler " [1]. The key change of the architecture is the design and implementation of the API for submitting jobs in the Jobs manager component, instead of previously assumed usage of an external queueing system.
In Figure 1 the actual architecture of the system is presented. The changes compared to architecture presented in [1] are highlighted in red.

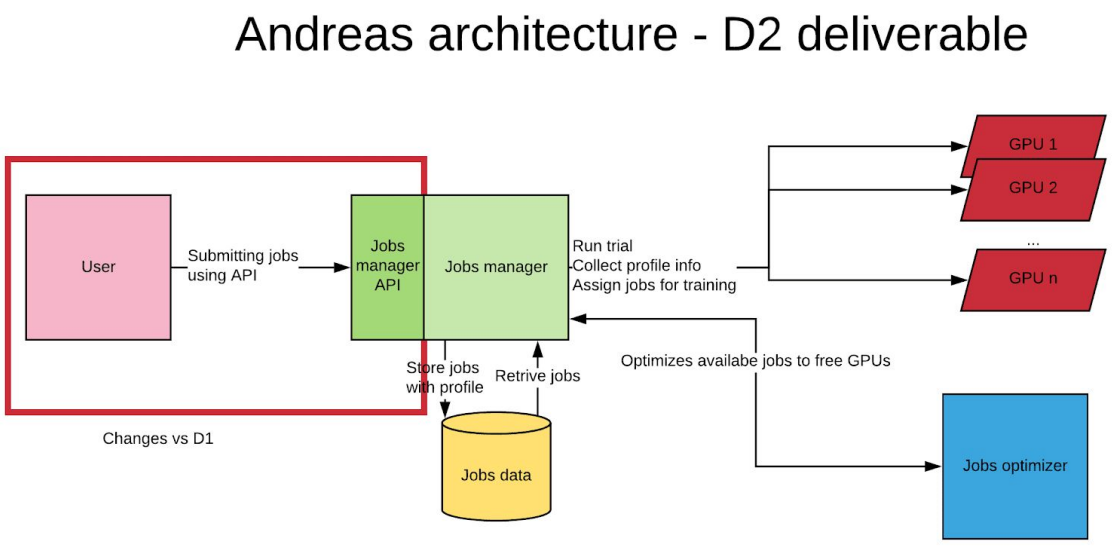

Figure 1. Updated ANDREAS high level architecture diagram

In Table 1 the actual list of components of the system is presented. The component Jobs queue described in deliverable D1 has been replaced by the Job manager API component.

Id	Component	Description	Type
1.	Jobs manager API	The API exposed by Jobs manager component to allow jobs submission.	Internal Andreas component
2.	Jobs manager	The component responsible for handling the workflow in ANDREAS.	Internal Andreas component
3.	Jobs optimizer	The component responsible for job's placement optimization.	Internal Andreas component
4.	Jobs data	Database to store jobs and profile information.	Internal Andreas component
5.	GPU	SLURM handled GPUs used to collect profile information and execute training.	External

Table 1. Updated list and description of ANDREAS components

The sequence is depicted on Figure 2 and described below. The main change is the usage of Jobs manager API to submit the job to the Andreas system instead of queuing approach.

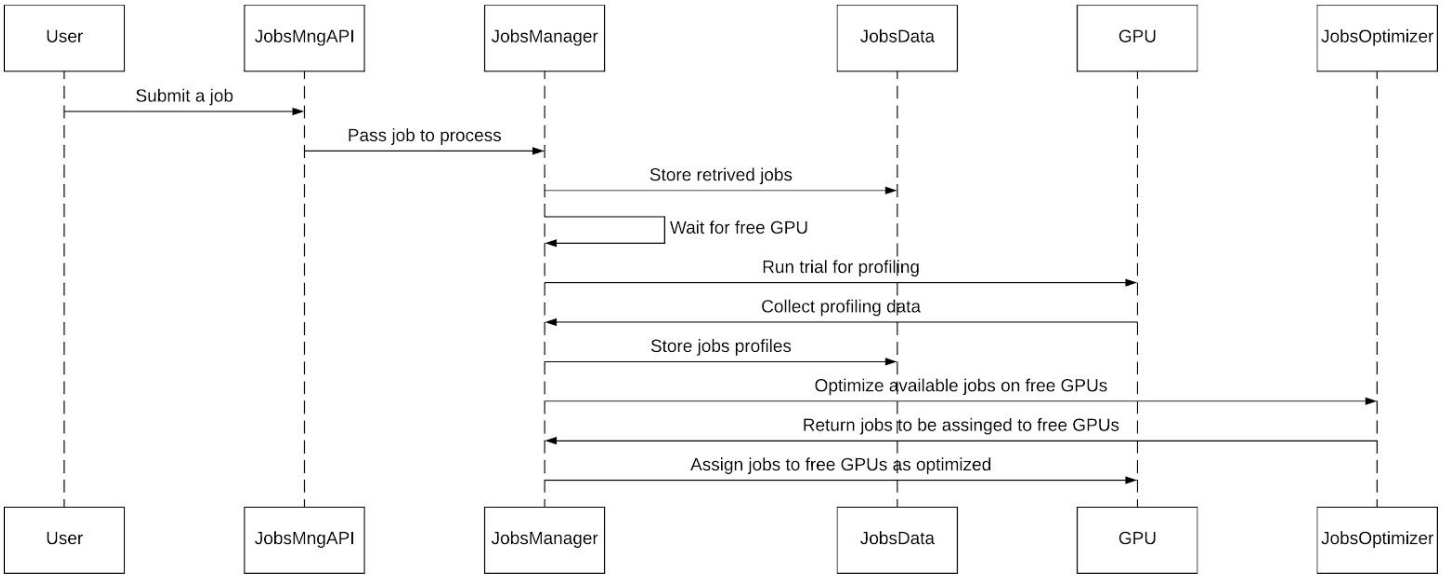

Figure 2. Updated ANDREAS sequence flow diagram

Description of the actual flow:

1. A new machine learning training Job is submitted using the Job manager API (changes compared to sequence described in [1]).
2. The Jobs Manager stores Job-related information in the JobsData database.
3. The Jobs Manager is waiting for the available GPU to start profiling operations.
4. The Jobs Manager is running Jobs to gather profiling information - according the profiling rules (currently per iteration execution time are gathered by running few training epochs on dedicated profiling node)
5. The Jobs Manager stores profiling data in JobsData database.
6. The Jobs Manager invokes Jobs Optimizer to optimize Jobs execution.
7. The Optimizer optimizes Jobs execution providing the number, type of GPU to be used to run the job within its deadline, minimizing energy consumption and job tardiness in case the deadline cannot be fulfilled. The Optimizer also identifies the node of the cluster where the Job (application) container has to be run.
8. The Jobs Manager runs Jobs according to the optimization results .

The structure of the database has been slightly changed for the profile information storing. The profile data are stored in each iteration instead of full training. The second change is a dedicated instance of the GPU for profiling. In the further release of the Andreas the more complex approach to profiling will be introduced to cover collecting profiling data for various GPU types.

The flow described above is implemented in the form of a second Andreas release and was tested with benchmarking applications (see also Section 7).

3. Integration with SLURM

SLURM (Simple linux utility for resource management) was installed on the Armida cluster to enable launching jobs on nodes and specifying resources that can be used by them. The maximum values of those resources had to be set during configuration of the SLURM.
Running jobs with SLURM proved to be faster than using bash commands via an established ssh connection. SLURM also enables simple monitoring of the processes and occupied CPUS and GPUS from outside the Jobs Manager. Such an additional capability was helpful during development work.

The command srun from SLURM was used to launch jobs:

srun --gres=gpu:tesla:1 --nodelist=armida-06 docker run ...

It indicates to run 'docker run' task using one GPU of type Tesla on node armida-06.
Information about a specific node and GPU was given based on the optimizer solution.The optimiser ensured that those resources will be available.
However, when Jobs Manager starts a profiling of the job it uses the node dedicated only for the profiling jobs. At this step we don't have an assurance that the resources needed for profiling will be free. In such a situation we rely on SLURM queue and abilities to schedule tasks by SLURM itself. In consequence Jobs Manager can proceed with the workflow depicted in Figure 2, leaving SLURM the responsibility to schedule and start each of the profiling tasks.

4. ARMIDA software updates

ARMIDA (ARM Infrastructure for the Development of Applications) is an experimental 8-node cluster located at E4 Computer Engineering's premises. Each node has 2xMarvell TX2 sockets (per socket: 64 cores Marvell TX2@2,2/2.5 GHz), 256 GB RAM, 1xMellanox IB 100Gb EDR, 1xNvidia Tesla V100-PCIE 32GB). ARMIDA is based on RedHat Enterprise Linux 8.0 (kernel 4.18), GNU 8.3.1. The installed software includes ARM Allinea Studio (Compiler + Forge), GCC 10, Nvidia HPC SDK, CUDA 11.

Specifically for the ANDREAS project, E4 developed and made available an optimized version of ExaMon on ARMIDA. ExaMon (Exascale Monitoring) [4, 5] is a highly scalable framework for the performance and energy monitoring of HPC servers. It consists of several components that include: a communication layer based on IoT protocols; A set of data collection agents that periodically measure and deliver physical and micro-architectural quantities; A storage layer implementing a distributed and scalable time-series database using NoSQL technologies; An application layer that exposes the collected data to different uses such as visualization, data analysis, machine learning and artificial intelligence.

5. Machine learning jobs docker image with GPU support

One of the main challenges that AI developers have to face while using ARM platforms (and more specifically the aarch64 architecture) is the shortage of supported frameworks. Indeed, only a few examples are currently available on container hub like Docker Hub and the Nvidia GPU Cloud (NGC) catalog. Moreover, these frameworks for the aarch64 architecture are currently targeted only on CPU cores or low-end GPUs, leading to an under-exploitation of the aarch64 architecture when compared to contemporary x86/64 platforms that can take advantage of high-end accelerators. To address these issues, and to leverage the most recent generation of aarch64-based products featuring low-power consumption, availability of high-end GPUs and related development tools, E4 developed and made available within ANDREAS "TF2-gpu on ARM", the TF2-gpu container, a Docker container which packages a Tensorflow 2 framework that can take advantage of Nvidia GPUs.

The core base of the TF2-gpu container are the recent release of Nvidia drivers and CUDA libraries specifically optimized for aarch64 architectures and the Tool-Solutions repository of the ARM Software GitHub project (https://github.com/ARM-software) maintained by the ARM community. The current version of the container (i.e. v. 1.1) is based on Ubuntu 18.04 and it ships the following libraries:
- CUDA libraries v. 11.1.1,
- cuDNN v. 8.0.5,
- ARM Performance Library v. 20.2.1,
- oneDNN v. 1.5.1,
- NumPY v. 1.17.1,
- SciPY v. 1.4.1,
- OpenCV v. 4.5.0,
- Tensorflow v. 2.3.0.

"TF2-gpu on ARM" was extensively tested on ARMIDA (specifically for the ThunderX2 T99XX CPUs and Nvidia Volta cards). "TF2-gpu on ARM"can be executed on all modern ARM64 processors with and without a GPU.
The TF2-gpu container is used as a base image for the jobs managed by Andreas project.

6. Jobs optimizer changes in 2nd release

This section describes changes in Jobs optimizer in second release of Andreas.

6.1 New proxy function

The baseline proxy function denoted in [1] as f_{OBJ}, and reported here,

$$\min f_{\mathrm{OBJ}} = \sum_{j \in \mathcal{J}} (\omega_j \tau_j + \rho \omega_j \hat{\tau}_j) + \sum_{\substack{j \in \mathcal{J} \\ n \in \mathcal{N}}} \pi_{jn} \alpha_{jn} \tag{1}$$

focuses on cost minimization during the current scheduling step. This approach provides high-quality local solutions but proves to be short-sighted in terms of the long-term scenario. Indeed, being conservative in the use of resources (favoring the cheapest available configuration), it runs the risk of increasing the pressure of jobs waiting to be processed to the point of having to compensate for them through costly configurations.

In order to enhance the results of our method, particularly in the new version that will be presented in the next section, we have developed a different proxy function to be used in place of f_{OBJ}. The new proxy function is:

$$\max \bar{f}_{\mathrm{OBJ}} = \sum_{j \in \mathcal{J}} \frac{M_j}{\pi_j + \omega_j \tau_j}. \tag{2}$$

It aims at favoring the schedules that use the resources most efficiently, pursuing a trade-off between maximizing the number of completed jobs and minimizing the operations costs. The maximum processing time M_j at the numerator is used as a scaling factor to weight the importance of all jobs in the queue, focusing the interest on long-running jobs. The denominator is composed of two terms: the deployment cost π_j of each job and the penalty cost $\omega_j\tau_j$ associated with the due date violation.

The alternative proxy function in Equation (2), denoted as $\bar{f}_{OBJ}$, partially addresses the limitations of f_{OBJ} by shifting the focus to resource efficiency: each term decreases as the processing time and the tardiness increase. Consequently, the model tries to keep these values small for all jobs. This alternative objective function provides the best results when applied within the path relinking strategy described in the following.

6.2 Path Relinking algorithm

The heuristic algorithm reported in [1] has been updated with the development of a path relinking procedure, aiming at enhancing the results obtained with the Randomized Greedy method. As an additional modification, the cost of the current scheduling step, that was evaluated in the previous version exploiting f_{OBJ}, is now computed by means of a generic proxy function f_P. This proxy function can identify either f_{OBJ} or $\bar{f}_{OBJ}$, giving rise to different scheduling policy strategies. The new version of the algorithm is reported below:

Algorithm 1

STEP#0 - PREPROCESSING

$T_c \leftarrow T_c + T$
$\mathcal{N}_O \leftarrow \emptyset$ ▷ $\mathcal{N}_O$: set of open nodes
for all $j \in J$ **do**
  if j was partially executed in previous steps **then**
    for all $n \in \mathcal{N},\ g \in \mathcal{G}_n$ **do**
      $t_{jng} \leftarrow t_{jng} \cdot (100 - \widehat{c_p}_j)/100$
      ▷ $\widehat{c_p}_j$: completion percentage
    end for
  end if
  $\Delta_j \leftarrow T_c + \min_{n,g}\{t_{jng}\} - d_j$
end for

STEP#1 - SCHEDULING

$\mathcal{S}^* \leftarrow$ RANDOMIZED_CONSTRUCTION($\mathcal{J}$, $\mathrm{MaxIt_{RG}}$)
$(S^*, c^*) \leftarrow$ PATH_RELINKING($\mathcal{S}^*$, $\mathrm{MaxIt_{PR}}$)

STEP#2 - POSTPROCESSING

for all $\nu \in \mathcal{N}_O$ **do**
  if there exist idle GPUs on ν **then**
    Assign the idle GPUs to the job with the highest speed-up
  end if
end for

Path relinking is normally used as intensification strategy, to enhance the results obtained through a randomized construction procedure. This is done by exploring trajectories connecting good-quality solutions with the goal of escaping local optima [2], [3].

As reported in Algorithm (1), we have adapted this general framework to our context, implementing a path relinking procedure that receives as input the set *S** of good-quality solutions, also called *elite solutions*, built through the randomized construction procedure presented in [1]. It extracts from *S** the candidate solution S_s with best value $f_P(S_s)$, generates and explores paths connecting S_s to the other elite solutions in *S**, to find better solutions by incorporating attributes of good quality solutions.

Algorithm 2 Path relinking procedure

```
1: function PATH_RELINKING(S*, MaxIt_PR)
2:     S_s ← solution in S* with best f_P(S_s)                ▷ S_s: source solution
3:     for all S_t ∈ S*, S_t ≠ S_s do                          ▷ S_t: target solution
4:         iter = 0
5:         while S_s ≠ S_t and iter < MaxIt_PR do
6:             M ← GET_MOVES(S_s, S_t)
7:             M_E ← ∅                                         ▷ M_E: set of explored moves
8:             (m*, c*) ← (empty move, f_P(S_s))
9:             for all m ∈ M do
10:                if m ∉ M_E then
11:                    (m*, c*) ← EXPLORE_STEP(S_s, S_t, m)
12:                    M_E ← M_E ∪ {m}
13:                end if
14:            end for
15:            if m* is not empty then
16:                S_s ← (S_s ∘ m*)
17:            end if
18:            iter ← iter +1
19:        end while
20:    end for
21:    return (S_s, f_P(S_s))
22: end function
```

The procedure, reported in Algorithm (2), is based on the concept of move, which denotes any atomic change that can be performed to move from the source solution S_s in the direction of a target solution $S_t \in S^*$, $S_t \neq S_s$. The new candidate solution obtained by applying a move *m* to the current solution S_s is denoted as $S_s \circ m$. In our algorithm, a sequence of moves is performed, either until S_t is reached or until a maximum number of iterations has been performed. In our context, a move from the source S_s to the target S_t is defined as a pair (j, n). Having denoted by n_s and n_t the nodes where job j is deployed in the source and in the target solution, respectively, the type of GPU available on n must be the same as the type of GPU available on n_t (and possibly different from the type of GPU available on n_s). The function GET_MOVES called at line 6 of Algorithm (2), having determined n_t and the number of GPUs g_t assigned to job j in the target solution S_t, checks whether this number is different from the one selected for job j in the source solution S_s and if the type of GPU available on n_t is different from the one available on n_s. In this case, the function looks for a node n that, in solution S_s, has the same GPU type as n_t and at least g_t free GPUs. If such a node exists, (j, n) is added to the set *M* of candidate moves. It is worth noting that either n_s or n can be empty, if the job was not executed in the source solution or is not executed in the target one.

Given the set *M* of feasible moves, the method explores all possible $m \in M$ looking for a move that yields a solution with a better objective function value than $f_P(S_s)$, denoted in line 8 by c*. The exploration of a move *m*, performed at line 11 of Algorithm (2), consists in applying *m* to the source solution S_s, generating a new candidate solution denoted by $S' = S_s \circ m$, and in evaluating the function f_P, obtaining $c = f_P(S')$. If the value c is better than the value c* (i.e., if the current move *m* yields an improvement), the value of the best move *m**, initially empty, is updated accordingly.

To perform a deeper exploration of the space around S_s, the function EXPLORE_STEP is implemented as follows. Instead of considering only the single move *m* given as input, it can proceed recursively by determining the new set *M′* of moves leading from $S' = S_s \circ m$ to the target solution S_t and explore all $m' \in M'$ to determine a new candidate solution $S'' = S' \circ m$. If $f_P(S'')$ has a better value with respect to c*, the best move *m** takes the value of *m*. This recursive step strongly affects the quality of the final solution returned by the algorithm, since it allows to explore a wider range of candidate moves and solutions. In particular, it may happen that a move *m* would be rejected if we were considering a single-step exploration because
$f_P(S') = f_P(S_s \circ m)$ did not provide a better outcome than c*. However, the new candidate solution S′ may lead in the second step to a new solution $S'' = S' \circ m'$ such that $f_P(S'')$ is better not only than $f_P(S')$ but also than c* and, in this case, the move *m* is accepted as new *m**.

Once the exploration of the set *M* has been completed, if *m** is not empty, which means that we have found a move providing a better outcome c*, the solution S is updated by applying m* (see line 16) and the algorithm proceeds with the next iteration, having as new source solution $S_s \circ m$.

6.2.1 Optimizer preliminary performance evaluation

Some preliminary tests of the effectiveness of the new Path Relinking method have been performed in the same framework described in [1], involving systems with a number of available nodes N varying from 10 to 100 and a number of submitted jobs J = 10N. We have generated multiple instances of the problem, exploiting the same inter-arrival generation schema involving four different scenarios: exponential inter-arrival times and high, low and mixed inter-arrival rates. The results and the comparison between this method and the previously proposed Randomized Greedy approach are reported in Figure 3 and Figure 4.

In particular, Figure 3 reports the ratio between the average total cost obtained with all the proposed methods and the average total cost of Earliest Deadline First which demonstrated to be the best first principle heuristic according to the considered jobs workloads. It is possible to notice that the Path Relinking method has better performance than the Randomized Greedy method in all the considered scenarios.

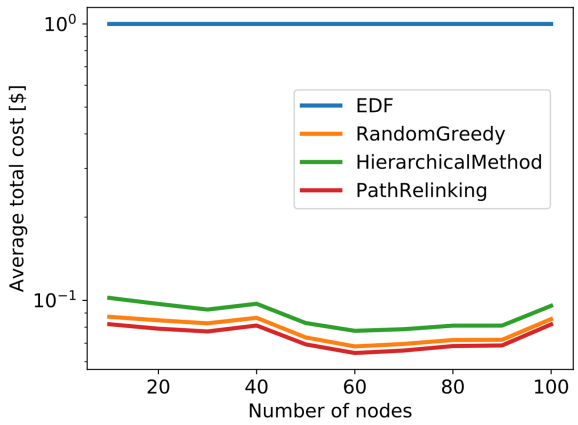

Figure 3.a - *exponential* inter-arrival times

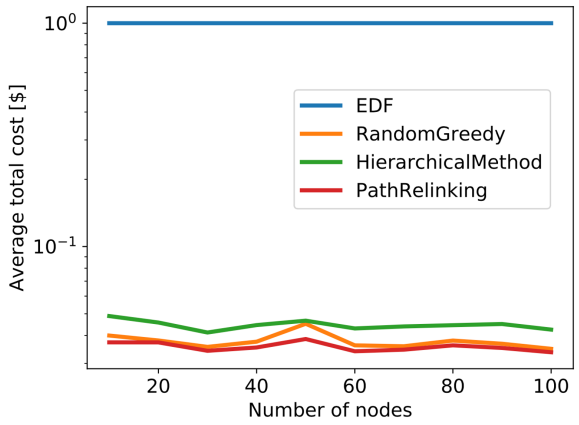

Figure 3.b - *high* inter-arrival rate

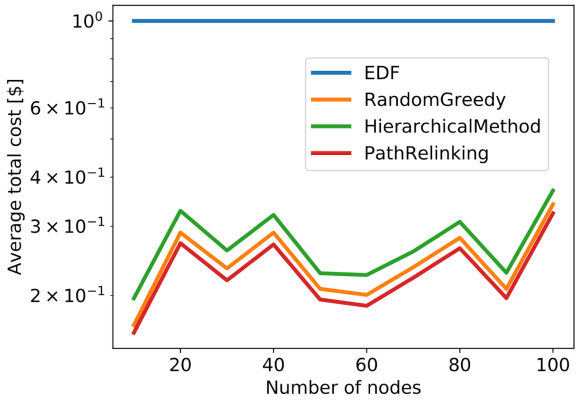

Figure 3.c - *mixed* inter-arrival rate

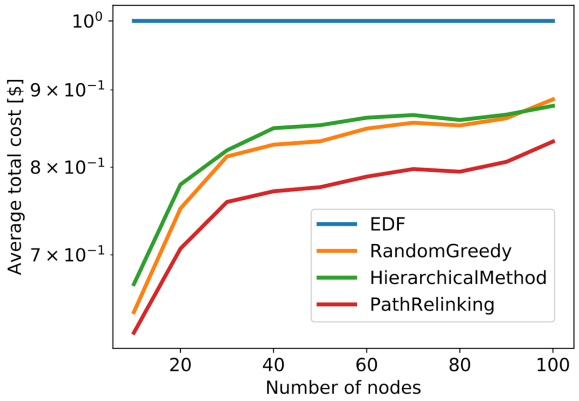

Figure 3.d - *low* inter-arrival rate

Figure 3 - Average total costs with the different methods

Computing the percentage cost reduction obtained by the Path Relinking approach over the Randomized Greedy approach, as reported in Figure 4, we can notice that the new method guarantees an average percentage gain of around 5% in all the considered scenarios. Moreover, it attains an average percentage gain between 7 and 23% with respect to the Hierarchical Method already used as benchmark in [1].

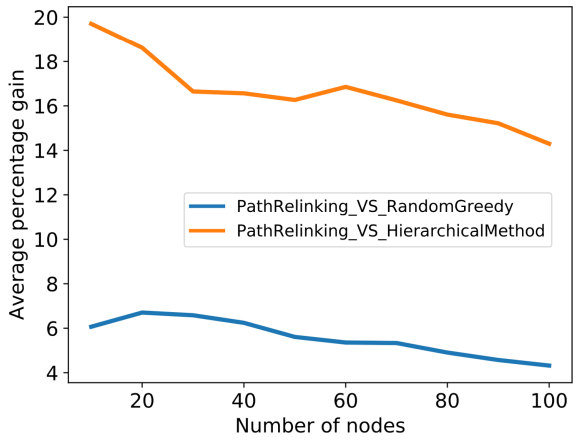

Figure 4.a - *exponential* inter-arrival times

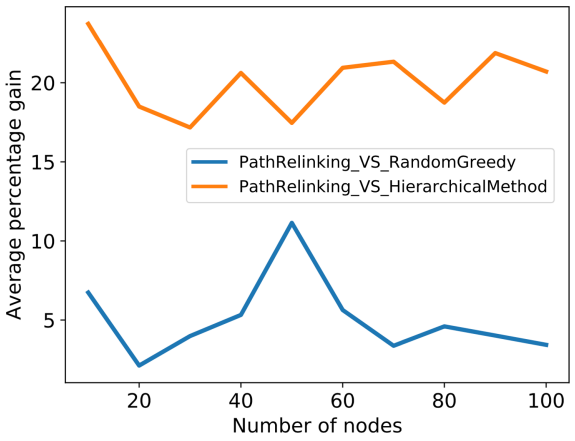

Figure 4.b - *high* inter-arrival rate

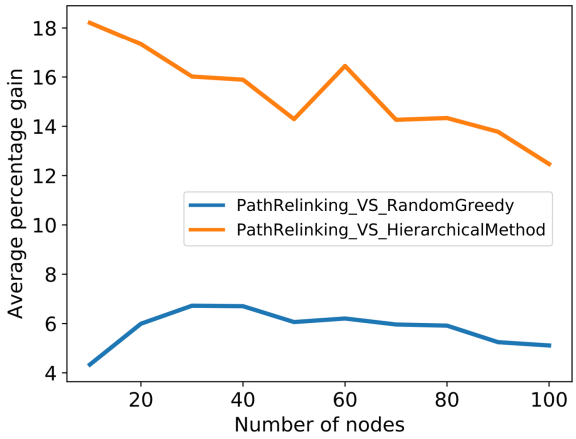

Figure 4.c - *mixed* inter-arrival rate

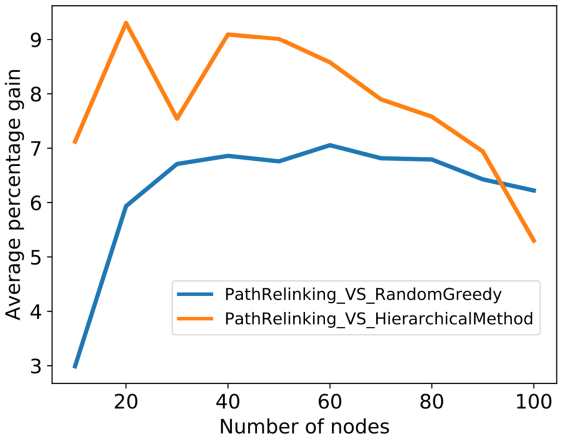

Figure 4.d - *low* inter-arrival rate

Figure 4 - Average percentage gain of Path Relinking method

Finally, we have reported in Figure 5 the average execution time of each call to the different methods. It is worth noticing that in all scenarios the Path Relinking method solves an instance of the problem with up to 100 nodes and 450 concurrent jobs in less than 7 seconds. Even if this time is slightly higher than what required by the Randomized Greedy approach, this still proves the feasibility of our method for practical scenarios.

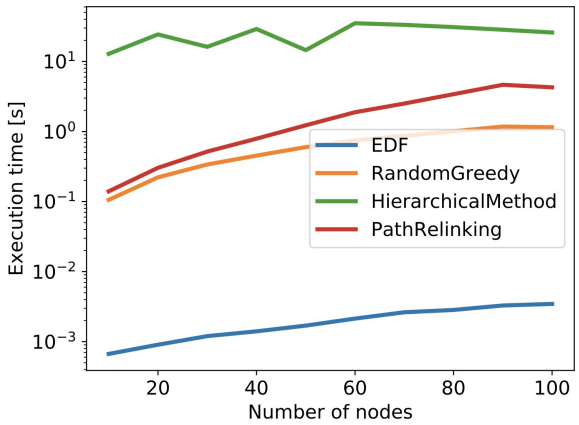

Figure 5.a - *exponential* inter-arrival times

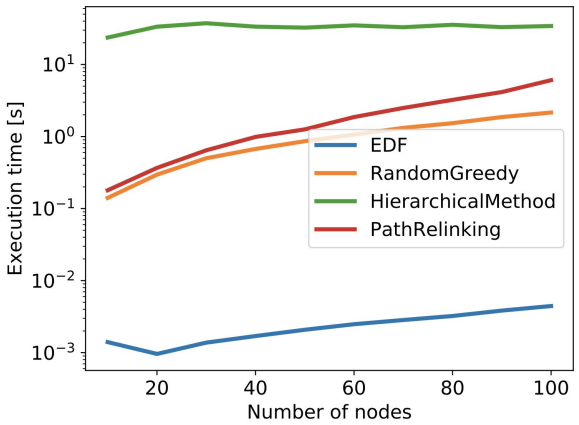

Figure 5.b - *high* inter-arrival rate

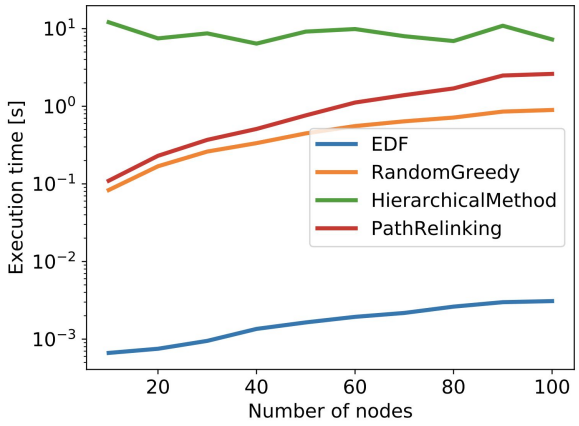

Figure 5.c - *mixed* inter-arrival rate

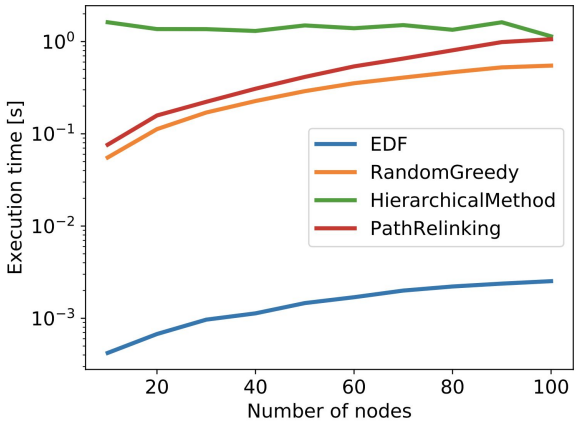

Figure 5.d - *low* inter-arrival rate

Figure 5. Average execution time of each optimizer call

7. Testing results

The section contains testing scenarios of the optimization scheduler that will be considered by the end of the project using the second release of the ANDREAS project product and reported in this deliverable.

The characteristics of the submitted jobs are summarized in Table 2. All of them are implemented relying on TensorFlow, exploiting the containers described in Section 5. The inter-arrival time has been set to 300s. The considered scenario has been accelerated, in terms of submission frequency and average execution times of applications. Nonetheless, this does not affect the results effectiveness since the workload assigned to the available nodes is comparable to practical situations. The system is composed of two nodes, whose characteristics in terms of type and number of GPUs are summarized in Table 3.

Job	Application	Files	Epochs	Batch size	Deadline	Priority
JJ1	DeepSpeech	64	158	4	3h	0.2
JJ2	VGG19	130,000	1	32	1h	0.2
JJ3	VGG19	130,000	1	32	2.8h	0.1
JJ4	AlexNet	130,000	12	256	1.5h	0.2
JJ5	ResNet50	130,000	3	64	2.3h	0.2
JJ6	ResNet50	130,000	3	64	1.5h	0.3

Table 2. Applications submitted to the system

Node ID	GPU type	Number of GPUs	Cost [$/h]
N1	NVIDIA V100	2	1.13
N2	NVIDIA V100	2	1.24

Table 3. Characteristics of the target nodes

The results obtained by the Path Relinking algorithm are reported in Figure 6. Jobs and the relative deadlines and priorities have been sized in order to highlight what may happen when resources are not available to run all jobs. Namely, in the fifth scheduling step, when job JJ5 is submitted, job JJ3, which is the one with lower priority, is preempted in order to free resources to run the new application. Also JJ5 is preempted, in the following scheduling step, in order to start JJ6, which has a higher priority. As a final consideration, the fact that job JJ3 is preempted makes it closer to its deadline. It is restarted in the eight scheduling step with only one GPU, since the other GPU available on N2 is assigned to JJ6, which has a higher priority. However, as soon as job JJ6 is completed, both GPUs available on N2 are assigned to JJ3 to complete it before its deadline.
Aim of these experiments is to validate our results by measuring the gap between real and predicted costs, including energy costs and penalties for deadline violations. The main expected deviations are due to a possible inaccuracy in jobs execution time estimate, as well as in the overhead produced when starting or restarting containers after a pre-emption.
We have considered a tentative list of models (see Table 2), which will be updated in the next period by including specific architectures particularly relevant for 7bulls customers, as LSTM and Recurrent Embedding Kernel. A final testing phase will be devoted to select the most representative networks, taking into account the ANDREAS project end-users' interests and requests.

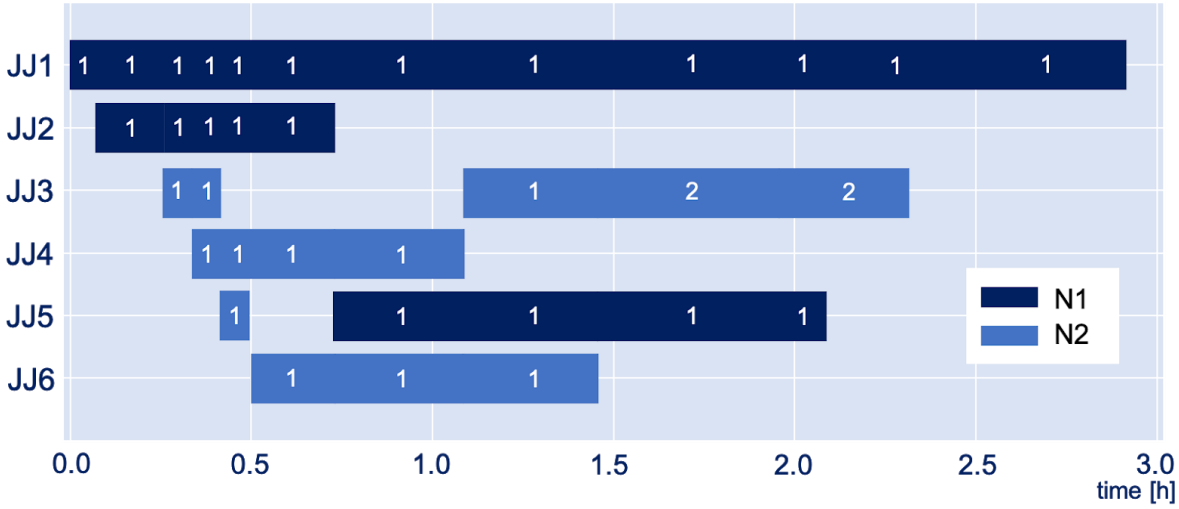

Figure 6. Results on the target system

8. Conclusion

This document describes the current status of the ANDREAS project development activities and summarizes changes compared to the first release.
In the final stage of the project we will focus on improving quality towards commercial use of the platform and further improve the ANDREAS scheduler to include feedback after second release testing and evaluation. Also, the extensive testing on both on-premises environments as well as for cloud ones will be performed as described in Section 7.

9. References

1. D. Ardagna et al. “D1 - 1st release of the integrated schedule”, 2020.
2. M. G. Resende, C. Ribeiro, “Grasp with path- relinking: Recent advances and applications,” in Meta-heuristics: Progress as Real Problem Solvers, T. Ibaraki, K. Nonobe, and M. Yagiura, Eds. Boston, MA: Springer US, 2005, pp. 29–63.
3. M. Resende, C. Ribeiro, “Greedy randomized adaptive search procedures: Advances and applications,” Handbook of Metaheuristics, Jan. 2010.
4. F. Beneventi, A. Bartolini, C. Cavazzoni, L. Benini, “Continuous learning of HPC infrastructure models using big data analytics and in-memory processing tools”, Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017 https://ieeexplore.ieee.org/abstract/document/7927143
5. A. Bartolin, F. Beneventi, A. Borghesi, D. Cesarini, A. Libri, L. Benini, C Cavazzoni, “Paving the Way Toward Energy-Aware and Automated Datacentre”, ICPP 2019: 48th International Conference on Parallel Processing, https://dl.acm.org/doi/abs/10.1145/3339186.3339215